Stochastic control problem: Multistage stochastic programing for portfolio optimization with derivatives

by

KARRI LAKSHMI YAMINI

2020MT60881

under the guidance of

Prof. ANANTA KUMAR MAJEE

A REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF MASTER IN TECHNOLOGY IN

Department of Mathematics

Indian Institute of Technology Delhi

June 2025

THESIS CERTIFICATE

This is to certify that the thesis titled *"Stochastic control problem: Multistage stochastic programing for portfolio optimization with derivatives"*, submitted by **Karri Lakshmi Yamini** (2020MT60881), to the Indian Institute of Technology Delhi, for the award of the degree of **Bachelor and Master of Technology**, is a bonafide record of the research work done by her under my supervision.

The contents of this thesis, in full or in parts, have not been submitted to any other institute or university for the award of any degree or diploma.

Dr. Ananta Kumar Majee
Assistant Professor
Department of Mathematics
Indian Institute of Technology Delhi

ACKNOWLEDGEMENTS

I extend my heartiest gratitude to my supervisor for having guided and encouraged me throughout the development of this thesis. I would also like to thank my family and friends for their constant and unwavering support.

Karri Lakshmi Yamini

Abstract

This thesis proposes a numerical framework for dynamic portfolio optimization under uncertainty using scenario trees and financial derivatives. Asset returns are modeled via bootstrapping and discretized using arbitrage-free scenario trees. European options are priced using the Minimal Entropy Martingale Measure (MEMM) to ensure risk-neutral consistency. Portfolio strategies are optimized using SLSQP under expected return and CVaR objectives. Results on real financial data show that incorporating options improves risk-adjusted performance and robustness.

Contents

1 Introduction and Thesis Overview **3**

1.1 Motivation and Context 3

1.2 Challenges in Financial Modeling 3

1.3 Contributions of this Work 4

1.4 Thesis Structure 5

2 Theoretical Foundations **6**

2.1 Stochastic Processes and SDEs 6

2.2 Stochastic Control and HJB Equations 8

2.2.1 Problem Setup and Wealth Dynamics 8

2.2.2 Objective Formulations 8

2.2.3 Remarks 9

2.3 Challenges in Continuous-Time Optimization 10

2.4 Discretization and Scenario Trees 11

2.5 Scenario Trees and Arbitrage-Free Modeling 11

2.5.1 Option Instruments and Payoff Modeling 17

2.5.2 Wealth Recursion with Option Cashflows 19

2.5.3 Portfolio Optimization Objectives 21

3 Implementation Framework **25**

3.1 Data Preprocessing and Return Computation 25

3.2 Scenario Tree Construction 26

3.2.1 Tree Structure and Stages 26

3.2.2 No-Arbitrage Filtering 26

3.2.3 Empirical Probability Assignment 26

3.2.4 Risk-Neutral Probability via MEMM 27
3.2.5 Resulting Tree 27
3.2.6 Option Instrument Creation and Pricing 28
3.2.7 Tree Flattening and Vectorization 30
3.2.8 Wealth Recursion and Propagation 32
3.2.9 Implementation Logic of Portfolio Optimization 34

4 Results and Analysis **36**
4.1 Risk-Neutral Probabilities via MEMM 36
4.2 Option Prices Across Scenario Tree 37
4.3 Optimization Results 37
4.3.1 Expected Wealth Maximization 37
4.3.2 With CVaR ($\alpha = 0.95$) 38
4.4 Interpretation of Results 38

A Python Source Code **39**

Chapter 1

Introduction and Thesis Overview

1.1 Motivation and Context

Imagine you are at an investment decision point — say, the start of a month. You don't know how the market will behave, but you can imagine several plausible future scenarios: markets may rise, fall, or stay flat, and each path leads to further possibilities in the next period. You want to choose an allocation strategy today that performs well across all these possible futures — maximizing returns, while also limiting losses in adverse cases.

This is the core idea behind **Multi-stage Stochastic Programming (MSP)** — a framework for modeling decision-making under multi-period uncertainty. Instead of relying on strong model assumptions, we discretize uncertainty using return samples from historical data to construct plausible market evolution paths.

Such a framework allows us to model dynamic portfolio strategies under uncertainty in a way that traditional static methods — like mean-variance optimization — cannot. It also opens the door to incorporating derivative instruments and risk-aware objectives, making the model more realistic and adaptable to actual market conditions.

1.2 Challenges in Financial Modeling

Despite the conceptual appeal, dynamic portfolio optimization in realistic settings presents several challenges:

- **Analytical Intractability:** Solving stochastic control problems or Hamilton-Jacobi-Bellman (HJB) equations becomes infeasible for high-dimensional portfolios

or in the presence of constraints.

- **Nonlinear Payoffs:** Instruments like options introduce path-dependent and asymmetric payoffs, which complicate the optimization process.
- **Scenario Sampling:** Modeling uncertainty through scenario trees requires careful sampling and filtering to avoid arbitrage, which could otherwise distort valuation and decisions.
- **Incomplete Markets:** In discrete-time models, perfect hedging is not always possible. This requires a robust pricing mechanism for derivatives, such as the Minimal Entropy Martingale Measure (MEMM).
- **Risk Sensitivity:** Pure return-based objectives may lead to overconfident or overly risky strategies. Incorporating downside risk measures like CVaR adds complexity but is essential for robust decision-making.

These challenges motivate a numerical approach that can balance tractability, realism, and robustness in modeling portfolio evolution and decision-making over time.

1.3 Contributions of this Work

This thesis presents a complete numerical framework for dynamic portfolio optimization under uncertainty, incorporating derivative instruments and risk-based objectives. The main contributions are:

- Develops an arbitrage-free scenario tree from historical return data using bootstrapping and filtering techniques.
- Incorporates European call and put options into the portfolio, allowing richer strategies and hedging capabilities.
- Prices derivatives using risk-neutral probabilities derived via the Minimal Entropy Martingale Measure (MEMM), ensuring market-consistent valuation.
- Models the evolution of wealth through recursive equations that integrate asset returns and option payoffs at each stage of the scenario tree.

- Formulates and solves portfolio optimization problems under expected return and CVaR-based objectives using constrained optimization techniques like SLSQP.
- Evaluates and compares strategies with and without derivative instruments using real financial data.

1.4 Thesis Structure

The thesis is organized as follows:

- **Chapter 2**: Reviews the mathematical background including SDEs, HJB equations, and scenario trees.
- **Chapter 3**: Describes the numerical framework, including tree generation, MEMM computation, option pricing, and optimization.
- **Chapter 4**: Presents experimental results from simulations and strategy comparisons.
- **Chapter 5**: Summarizes key findings and discusses future directions.

Chapter 2

Theoretical Foundations

2.1 Stochastic Processes and SDEs

Why Do We Need Stochastic Models in Finance?

As discussed in (14), stochastic differential equations (SDEs) are essential to capture the randomness in financial markets. Financial markets are inherently uncertain — prices move due to news, investor sentiment, macroeconomic events, and random fluctuations. Modeling such systems with deterministic equations (like ODEs) is too simplistic and fails to reflect reality.

Consider two investors trying to predict an asset's future value. If they assume the return is constant, they will both reach the same prediction. But in reality, asset prices rarely follow fixed trends. Hence, we need models that incorporate **randomness** — this is where **stochastic processes** come in.

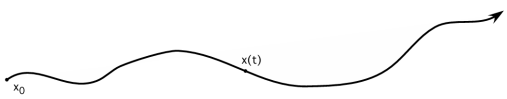

Figure 2.1: *
(a) Trajectory of an Ordinary Differential Equation (ODE)

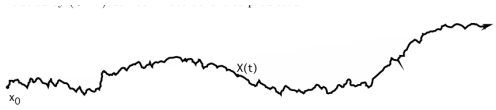

Figure 2.2: *
(b) Sample Path from a Stochastic Differential Equation (SDE)

Figure 2.3: While ODEs model deterministic price evolution, SDEs better capture the randomness in financial markets. This comparison illustrates the motivation behind using stochastic models for asset price dynamics.

Introduction to Stochastic Processes

A **stochastic process** is a collection of random variables indexed over time, used to describe systems that evolve with uncertainty. In finance, these are used to model prices, interest rates, volatilities, etc.

One of the most fundamental examples is the **Brownian motion** (or Wiener process), denoted W_t, which satisfies:

- $W_0 = 0$
- W_t has independent increments
- $W_t - W_s \sim \mathcal{N}(0, t-s)$ for $0 \leq s < t$
- W_t has continuous paths with probability 1

Stochastic Differential Equations (SDEs)

Stochastic processes are incorporated into dynamic models through **Stochastic Differential Equations (SDEs)**. A general SDE looks like:

$$dX_t = \mu(X_t, t)\, dt + \sigma(X_t, t)\, dW_t,$$

Where:

- X_t: the state variable (e.g., asset price),
- μ: deterministic drift (mean return),
- σ: volatility (how random it is),
- W_t: Brownian motion

These properties form the foundation of continuous-time stochastic modeling, especially in financial applications (15).

2.2 Stochastic Control and HJB Equations

2.2.1 Problem Setup and Wealth Dynamics

Consider a self-financing portfolio composed of m risky assets (e.g., stocks) and a risk-free asset (e.g., a bond). Let $X(t)$ denote the portfolio wealth at time t, and let $\omega_i(t)$ represent the monetary amount allocated to the i-th asset.

The price dynamics of the risky assets are modeled by:

$$dP_i(t) = P_i(t)\left[b_i(t)\,dt + \sum_{j=1}^{m} \sigma_{ij}(t)\,dW_j(t)\right], \quad dP_0(t) = r(t)P_0(t)\,dt$$

where $b_i(t)$ is the expected return of asset i, $\sigma_{ij}(t)$ is the volatility, and $r(t)$ is the risk-free rate.

The resulting wealth dynamics for the entire portfolio follow the SDE:

$$dX(t) = \left[r(t)X(t) + \sum_{i=1}^{m}(b_i(t) - r(t))\omega_i(t)\right]dt + \sum_{i=1}^{m}\sum_{j=1}^{m} \sigma_{ij}(t)\omega_i(t)\,dW_j(t)$$

This formulation captures how portfolio wealth evolves over time as a function of asset returns and allocation decisions (8).

2.2.2 Objective Formulations

Let the wealth invested in asset i at time t be given by:

$$\omega_i(t) = N_i(t) \cdot P_i(t)$$

where $N_i(t)$ is the number of units held. Let μ_i denote the expected return of asset i. Based on the investor's preference, the typical control objectives are:

1. **Risk Minimization:**

$$\min_{\omega(t)} \mathbb{E}\left[\int_0^T \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m} \omega_i(t)\sigma_{ij}(t)\omega_j(t)\,dt\right]$$

2. **Expected Return Maximization:**

$$\max_{\omega(t)} \mathbb{E}\left[\int_0^T \left(\sum_{i=1}^m \mu_i \omega_i(t)\right)^2 dt\right]$$

These objectives give rise to a continuous-time stochastic control problem where $\omega(t)$ is the control variable over the investment horizon.

2.2.3 Hamilton–Jacobi–Bellman (HJB) Formulation

An alternative approach to solving the above control problem is via dynamic programming, which leads to the Hamilton–Jacobi–Bellman (HJB) equation (17). Let $V(x,t)$ denote the value function representing the optimal expected utility starting from wealth x at time t. The HJB equation is:

$$\frac{\partial V}{\partial t} + \sup_{\omega}\left\{f(x,\omega,t) + \mu(x,\omega,t)\frac{\partial V}{\partial x} + \frac{1}{2}\sigma^2(x,\omega,t)\frac{\partial^2 V}{\partial x^2}\right\} = 0$$

with terminal condition:

$$V(x,T) = g(x)$$

Here:

- $f(x,\omega,t)$ is the running reward function (e.g., returns),
- $g(x)$ is the terminal utility or wealth goal at time T.

This nonlinear PDE captures the dynamic trade-off between return and risk, and solving it yields the optimal control $\omega^*(t)$ for dynamic portfolio management.

2.2.3 Remarks

- Stochastic control provides the foundation for solving continuous-time portfolio optimization. - The HJB framework ensures time-consistent strategies. - These analytical results serve as theoretical benchmarks, but in practice, we discretize and solve them using stochastic programming (MSP) and numerical optimization methods (e.g., SLSQP), which will be elaborated in upcoming chapters.

2.3 Challenges in Continuous-Time Optimization

While the stochastic control and HJB framework offers elegant theoretical tools, it faces several practical challenges in real-world portfolio optimization (10):

1. High Dimensionality

Realistic portfolios typically involve multiple risky assets. Solving the HJB equation in such high-dimensional spaces becomes computationally infeasible due to the **curse of dimensionality** — the exponential growth in computation as the number of state variables increases.

2. Nonlinearity and Constraints

Portfolio allocation is subject to a variety of constraints:

- No short-selling or leverage limits,
- Allocation caps on individual assets,
- Liquidity and transaction costs,
- Inclusion of derivatives with nonlinear payoffs.

These factors introduce nonlinearity and discontinuities into the control problem, making closed-form or even PDE-based solutions impractical.

3. Market Incompleteness and Derivative Pricing

In complete markets, every contingent claim (like a European option) can be perfectly replicated using a portfolio of underlying assets. But real markets are typically **incomplete**, due to discrete trading, limited instruments, and transaction costs (18).

This makes perfect replication and arbitrage-free pricing impossible using classical methods. To handle this, we adopt the **Minimal Entropy Martingale Measure (MEMM)** — a principled way to assign risk-neutral probabilities closest (in a KL-divergence sense) to the observed distribution while ensuring no-arbitrage.

4. Risk-Sensitive Objectives

Objective functions based solely on expected return ignore downside risk. To achieve robust performance across scenarios, we incorporate coherent risk measures such as **Conditional Value-at-Risk (CVaR)**. However, incorporating such measures adds computational and structural complexity **(author?)** (19).

Summary and Discretization Approach

These challenges render continuous-time methods analytically elegant but impractical. As a remedy, we shift to a discrete-time framework using **Multi-Stage Stochastic Programming (MSP)**:

- **Uncertainty** is modeled via a scenario tree, constructed from historical return samples.
- **Derivatives** are priced using MEMM-based risk-neutral probabilities at each node.
- **Optimization** is performed numerically over a finite decision horizon using constrained solvers (e.g., SLSQP).

This discretization allows us to realistically capture market dynamics, path-dependent strategies, and practical constraints — making portfolio optimization both tractable and realistic. This forms the computational and theoretical basis of our work — combining scenario generation, MEMM-based valuation, and portfolio optimization under return and risk objectives. The full numerical framework is developed in the next chapter.

2.4 Discretization and Scenario Trees

2.5 Scenario Trees and Arbitrage-Free Modeling

Motivation: Discrete Decision-Making Under Uncertainty

In real-world finance, investment decisions are made sequentially under uncertainty. At each point in time, investors face unknown future market outcomes. While continuous-time models like SDEs offer theoretical elegance, solving them analytically becomes infeasible for multi-asset portfolios with realistic constraints.

To make the problem computationally tractable, we adopt a **discrete-time, scenario-based** framework. Instead of modeling asset returns as continuous processes, we approximate them using a finite set of possible outcomes at each time step. This leads to the construction of a **scenario tree** — a fundamental structure for dynamic decision-making under uncertainty(10).

What is a Scenario Tree?

A scenario tree is a discrete representation of market uncertainty across multiple stages. Each node in the tree represents a specific market state at a given time, and branches between nodes represent plausible return paths to future states(10).

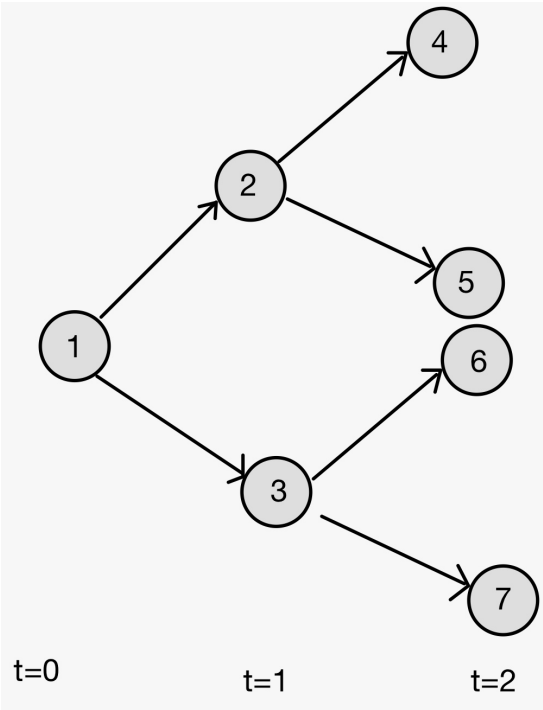

Figure 2.4: An illustrative 2-stage scenario tree capturing evolving market states. Each edge denotes an asset return vector.

Formally, let $\mathcal{T}$ be the set of nodes in the scenario tree. Each node $n \in \mathcal{T}$ has:

- A time index $t(n) \in \{0, 1, \ldots, T\}$,
- A parent node $\pi(n)$ (except for the root node),
- A set of child nodes $\mathcal{C}(n)$,

- A return vector $r(n) \in \mathbb{R}^d$, where d is the number of financial instruments (e.g., stock, bond, option),
- A probability $p(n)$ under the real-world measure $\mathbb{P}$.

Node Representation Example

Let us consider an example for **Node 2** (see Figure 2.4):

- **Parent:** $\pi(2) = 1$ (Node 1 is the root node at $t = 0$),
- **Children:** $\mathcal{C}(2) = \{4, 5\}$ (future scenarios at $t = 2$),

Step 1: Building the Tree via Bootstrapping

We use a non-parametric **bootstrapping** method:

- From each node, sample b return vectors from historical data.
- Create b child nodes using these returns.
- Repeat for T stages to construct the full tree.

This process captures return behavior without relying on parametric assumptions.

Step 2: Ensuring Arbitrage-Free Modeling

What is Arbitrage? Arbitrage refers to the opportunity of earning a guaranteed profit with no risk. In a scenario tree, arbitrage exists if a portfolio of risky assets yields more than the risk-free asset in all possible future outcomes(20).

How do we detect arbitrage? At any parent node (e.g., Node 1 in Figure 2.5), suppose we observe $b = 3$ child nodes, each associated with a return vector for $d = 3$ assets (including risk-free). Let the parent node return vector at stage $k - 1$ be:

$$\mathbf{r}_{0,k-1} = \begin{bmatrix} r_{11,k-1} \\ r_{21,k-1} \\ r_{31,k-1} \end{bmatrix}$$

and the return vectors of the 3 children at stage k be:

$$\mathbf{r}_{1,k} = \begin{bmatrix} r_{12,k} \\ r_{22,k} \\ r_{32,k} \end{bmatrix}, \quad \mathbf{r}_{2,k} = \begin{bmatrix} r_{13,k} \\ r_{23,k} \\ r_{33,k} \end{bmatrix}, \quad \mathbf{r}_{3,k} = \begin{bmatrix} r_{14,k} \\ r_{24,k} \\ r_{34,k} \end{bmatrix}$$

We check whether there exist weights (probabilities) $p_1, p_2, p_3 \geq 0$ such that:

$$\mathbf{r}_{0,k-1} = p_1 \cdot \mathbf{r}_{1,k} + p_2 \cdot \mathbf{r}_{2,k} + p_3 \cdot \mathbf{r}_{3,k}, \quad \text{with } p_1 + p_2 + p_3 = 1$$

This is equivalent to checking whether the parent return vector is a convex combination of future return vectors. If such probabilities exist → **no arbitrage**.

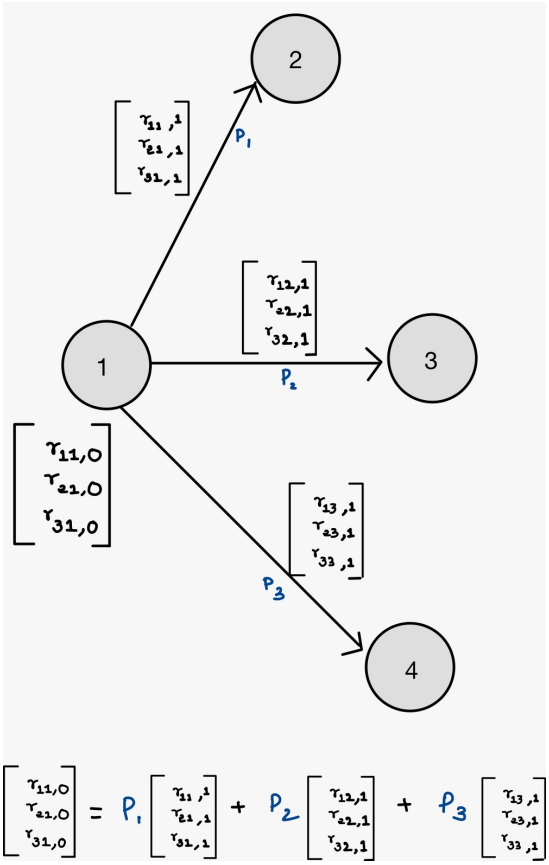

Figure 2.5: Martingale check: Can the parent return vector be written as a convex combination of its children?

Step 3: Assigning Empirical Probabilities

Once the tree is ensured to be arbitrage-free, we assign **empirical probabilities** to each node. These reflect the likelihood of reaching that node under a sampling scheme (e.g., uniform bootstrapping)(10).

Notation:

- Let $p(n)$: empirical probability of node n,
- Let $\pi(n)$: parent of node n,
- Let b: branching factor (number of child nodes per parent),
- Let T: depth of the scenario tree.

Recursive Assignment:

$$p(n) = \frac{1}{b} \cdot p(\pi(n)), \quad \text{with } p(\text{root}) = 1$$

Path Probability: Each complete path ω from root to leaf has probability:

$$\mathbb{P}(\omega) = \left(\frac{1}{b}\right)^T$$

Usage: These empirical probabilities are used:

- To compute expected wealth in portfolio optimization,
- As base measure $\mathbb{P}$ in MEMM for risk-neutral pricing.

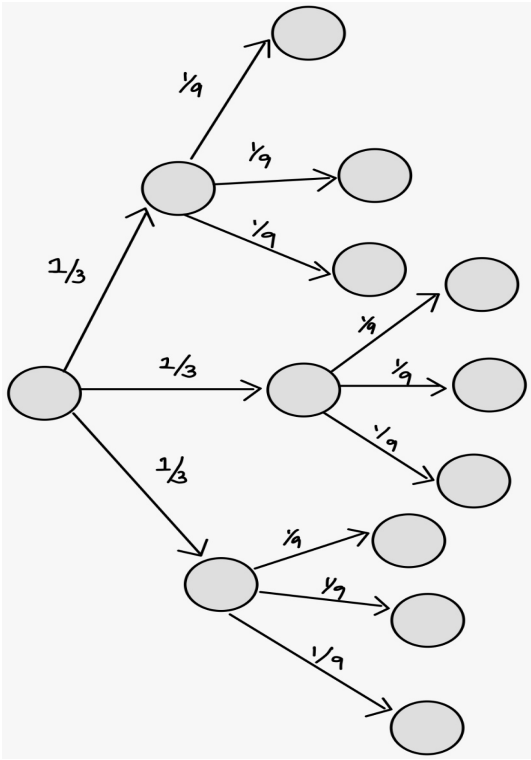

Figure 2.6: Illustration of empirical probability assignment in scenario tree with three child nodes

—

Step 4: Risk-Neutral Pricing via Minimal Entropy Martingale Measure (MEMM)

Why not use $\mathbb{P}$ for pricing? In our framework, $\mathbb{P}$ represents the empirical measure assigned to scenario tree paths based on historical sampling (e.g., uniform bootstrapping). While this measure reflects plausible real-world evolution of returns, it does not guarantee that discounted asset prices follow a martingale process — a key requirement for arbitrage-free derivative pricing. Therefore, we instead construct a risk-neutral measure $\mathbb{Q}$, under which discounted asset prices evolve as martingales. This transformation is performed using the Minimal Entropy Martingale Measure (MEMM), which selects the closest no-arbitrage-consistent measure $\mathbb{Q}$ to the empirical $\mathbb{P}$ in terms of relative entropy (18).

No-Arbitrage Martingale Condition: Given child return vectors $\mathbf{r}_{j,k}$ at stage k, and parent vector $\mathbf{r}_{0,k-1}$, we want:

$$\mathbf{r}_{0,k-1} = \sum_{j=1}^{b} q_j \cdot \mathbf{r}_{j,k} \quad \text{with} \quad \sum_{j=1}^{b} q_j = 1, \quad q_j \geq 0$$

This ensures that current returns are explained as the expected future returns under risk-neutral probabilities $q_j \in \mathbb{Q}$ — enforcing martingale and no-arbitrage.

MEMM Optimization Problem: We choose the $\mathbb{Q}$ that is closest to $\mathbb{P}$, minimizing relative entropy (Kullback–Leibler divergence):

$$\mathbb{Q}^* = \arg\min_{\mathbb{Q}\in\mathcal{M}} \sum_{j=1}^{b} q_j \log\left(\frac{q_j}{p_j}\right)$$

subject to:

- $\sum_{j=1}^{b} q_j = 1$ (valid probability vector),
- $q_j \geq 0 \quad \forall j$,
- $\mathbf{r}_{0,k-1} = \sum_{j=1}^{b} q_j \cdot \mathbf{r}_{j,k}$ (martingale condition).

Interpretation: MEMM picks the **least biased** transformation from real-world to risk-neutral probabilities — one that allows arbitrage-free derivative pricing while staying close to historical data.

2.5.1 Option Instruments and Payoff Modeling

In addition to traditional equity and risk-free assets, our model incorporates European options as financial derivatives to allow for more flexible and effective portfolio hedging strategies. These options are priced and valued at each node of the scenario tree and their associated cashflows are directly integrated into the investor's wealth dynamics (16).

Option Definition

Each European option is characterized by:

- **Strike Price** K,
- **Maturity Stage** T_o, where $T_o \leq T$,

- **Type** — either *call* or *put*,
- **Underlying Asset Index** i, indicating the asset on which the option is written.

The payoff at maturity for a node $n \in \mathcal{T}$ at stage $t(n) = T_o$ is computed as:

$$\text{Call Option:} \quad \max(S_i(n) - K, 0)$$

$$\text{Put Option:} \quad \max(K - S_i(n), 0)$$

where $S_i(n)$ denotes the price of asset i at node n, computed via compounded returns from the root node:

$$S_i(n) = S_i(n_0) \cdot \prod_{k=1}^{t(n)} (1 + r_i(n_k))$$

Here, $\{n_k\}$ is the path from the root node to node n, and $r_i(n_k)$ is the return of asset i on that branch (10).

Backward Pricing with Risk-Neutral Probabilities

To determine the fair value of an option at any node prior to maturity, we employ risk-neutral valuation using the Minimal Entropy Martingale Measure (MEMM). Starting from the terminal payoffs at stage T_o, we apply a backward recursion using risk-neutral probabilities $\mathbb{Q}$ computed for each node (21):

$$V_j(n) = \sum_{c \in \mathcal{C}(n)} \mathbb{Q}(n \to c) \cdot V_j(c)$$

where:

- $V_j(n)$ is the value of option j at node n,
- $\mathcal{C}(n)$ is the set of children of node n,
- $\mathbb{Q}(n \to c)$ is the MEMM-based transition probability from n to child c.

This backward recursion ensures that the computed option values are consistent with no-arbitrage pricing and reflect market-consistent expectations under the risk-neutral measure.

Economic Interpretation

The inclusion of options allows the investor to hedge against downside risks and exploit asymmetric return opportunities. For instance, in bear or volatile markets, put options may protect against large losses, while call options allow participation in upward trends. When optimally priced and allocated, options can significantly improve the risk-adjusted performance of the portfolio.

2.5.2 Wealth Recursion with Option Cashflows

We now describe how the investor's wealth evolves through the scenario tree, accounting for both asset returns and derivative positions. At each node n, wealth is updated recursively from the parent node $\pi(n)$ based on investment returns and cashflows from options (10).

Notations

- $W(n)$: wealth at node $n \in \mathcal{T}$,
- $x_i(\pi(n)) \in [0,1]$: proportion of wealth invested in asset $i \in \{1,\dots,d\}$ at parent node $\pi(n)$,
- $r_i(n)$: return of asset i from $\pi(n)$ to n,
- $y_j(\pi(n)) \in [0,1]$: proportion of wealth invested in option $j \in \{1,\dots,J\}$ at node $\pi(n)$,
- $V_j(n)$: MEMM-based value of option j at node n,
- τ_j: maturity stage of option j,
- $t(n)$: stage index of node n,
- $\text{Payoff}_j(n)$: intrinsic payoff of option j at node n, applicable only when $t(n) = \tau_j$.

Option Holdings and Units

The number of units of option j purchased at node $\pi(n)$ is given by:

$$v_j(\pi(n)) = \frac{y_j(\pi(n)) \cdot W(\pi(n))}{V_j(\pi(n))}$$

This ensures that $y_j(\pi(n))$ is interpreted as the proportion of wealth allocated to option j, and $v_j(\pi(n))$ gives the actual number of options held.

Option Cashflow Term

At any child node n, the option cash contribution is:

$$C(n) = \sum_{j=1}^{J} v_j(\pi(n)) \cdot \begin{cases} \text{Payoff}_j(n), & \text{if } t(n) = \tau_j \quad \text{(maturity)} \\ V_j(n), & \text{otherwise} \end{cases}$$

Wealth Recursion Equation

Putting it all together, the wealth at any node n is computed as:

$$W(n) = W(\pi(n)) \cdot \left(\sum_{i=1}^{d} x_i(\pi(n)) \cdot (1 + r_i(n)) \right) + C(n)$$

The first term represents the capital gains from invested assets, and the second term $C(n)$ captures the cash inflow from options, either through payoff (at maturity) or mark-to-market revaluation.

Economic Interpretation

This recursive structure accounts for:

- Realistic capital appreciation from risky and risk-free assets,
- Dynamic revaluation and maturity of derivative positions,
- Correct scaling of option units based on previous wealth,
- Time-consistent wealth propagation in the scenario tree.

It enables the optimization model to capture both market exposure and derivative-driven hedging effects in a unified framework.

2.5.3 Portfolio Optimization Objectives

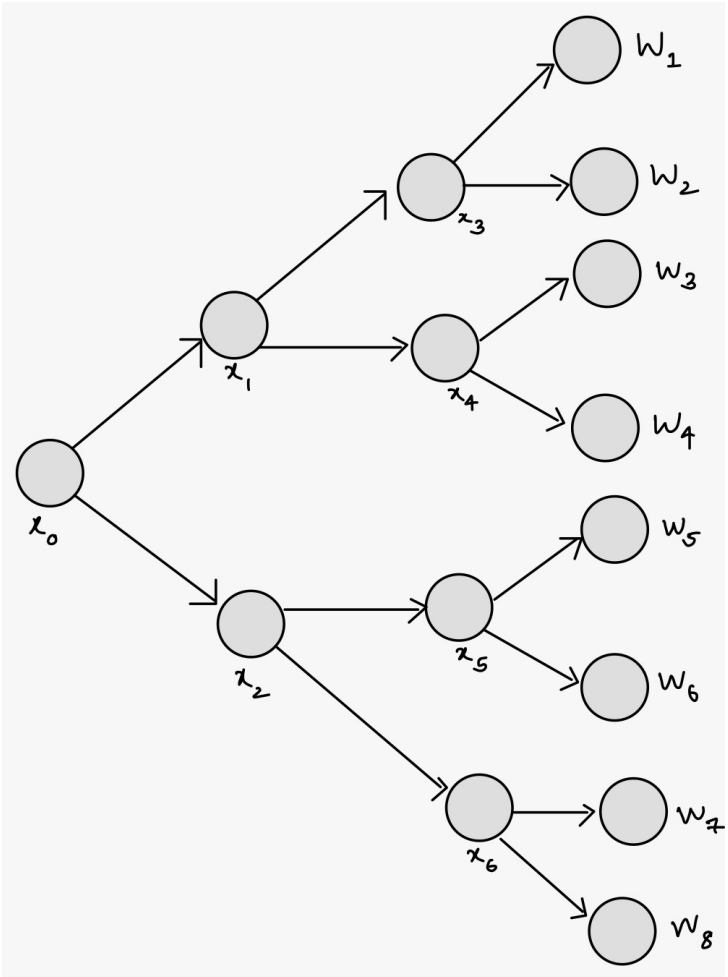

Figure 2.7: Scenario tree showing allocation decisions x_i at different nodes and resulting terminal wealth outcomes W_j. These decisions determine how wealth propagates through various market paths.

The scenario tree in Figure 2.7 illustrates how portfolio allocation decisions x_i at each node influence the evolution of wealth across different market paths. Each branch represents a possible market return, and each decision variable x_i corresponds to an allocation made at that state.

Our goal is to make these allocation decisions in such a way that they collectively **maximize the expected terminal wealth** at the leaf nodes W_j.

After modeling the dynamics of asset returns and option cashflows through the scenario tree, our next goal is to determine the optimal portfolio strategy that governs allocation decisions at each node. These decisions directly influence the investor's final wealth at the terminal nodes.

Wealth Propagation and Terminal States

Let $W(n)$ denote the wealth at a node $n \in \mathcal{T}$, propagated recursively as described in Section 2.5.2. At the final stage T, we obtain a set of terminal nodes $\mathcal{L} \subset \mathcal{T}$, each representing a complete scenario path of market evolution and investment decisions.

The terminal wealth $W(n)$ at each node $n \in \mathcal{L}$ depends on:

- All allocation decisions taken along the path from the root to n,
- The realized asset returns along this path,
- Cashflows from options exercised or held during the path.

Objective 1: Expected Wealth Maximization

The most straightforward objective is to maximize the expected wealth at the end of the investment horizon. Since all scenario paths are generated using a uniform sampling mechanism (e.g., bootstrapping with equal weights), we assume equal real-world probability $p(n) = 1/|\mathcal{L}|$ for each terminal node $n \in \mathcal{L}$.

Thus, the objective becomes:

$$\max_{x,y} \quad \mathbb{E}_{\mathbb{P}}[W] = \frac{1}{|\mathcal{L}|} \sum_{n \in \mathcal{L}} W(n)$$

subject to:

$$\sum_{i=1}^{d} x_i(n) + \sum_{j=1}^{J} y_j(n) = 1, \quad x_i(n), y_j(n) \geq 0 \quad \forall n \in \text{decision nodes}$$

This approach rewards strategies that perform well on average but does not consider the variability of outcomes across scenarios.

Need for Risk Awareness: Motivation for CVaR

Maximizing expected wealth alone can lead to strategies that exploit a few high-return scenarios while exposing the investor to severe losses in adverse outcomes. If the distribution of terminal wealth is highly skewed — with a few large values and many low ones — the average may be misleading.

To address this, we incorporate a risk-aware objective using the **Conditional Value-at-Risk (CVaR)**, which captures the expected loss in the worst-case scenarios. CVaR provides a coherent measure of downside risk and ensures that the strategy is robust against poor market conditions.

Objective 2: CVaR Minimization

Let $\alpha \in (0, 1)$ be the risk level. Define the CVaR of terminal wealth as (19):

$$\mathrm{CVaR}_\alpha(W) = \frac{1}{\alpha} \sum_{n \in \mathcal{L}_\alpha} \frac{1}{|\mathcal{L}|} W(n)$$

where $\mathcal{L}_\alpha \subset \mathcal{L}$ denotes the worst $\alpha \cdot |\mathcal{L}|$ wealth outcomes.

Then the risk-averse optimization becomes:

$$\min_{x,y} \quad \mathrm{CVaR}_\alpha(W)$$

subject to the same allocation and feasibility constraints.

This objective prioritizes minimizing potential losses in the tail of the wealth distribution, ensuring conservative and stable strategies.

Objective 3: Risk-Return Trade-off

In practice, investors typically seek a balance between return and risk. This leads to a convex combination of the previous two objectives:

$$\max_{x,y} \quad (1 - \lambda) \cdot \mathbb{E}_{\mathbb{P}}[W] - \lambda \cdot \mathrm{CVaR}_\alpha(W) \quad \text{for } \lambda \in [0, 1]$$

This unified formulation allows for flexible tuning:

- $\lambda = 0$: pure return maximization (risk-neutral),
- $\lambda = 1$: pure CVaR minimization (risk-averse),
- $0 < \lambda < 1$: balanced risk-reward trade-off.

Interpretation and Practical Implications

By computing wealth across the scenario tree and choosing decisions that shape its evolution, we construct investment policies that are not just forward-looking, but also risk-controlled. The choice of objective allows us to align strategies with investor preferences — whether return-seeking, risk-averse, or balanced.

In the experiments that follow, we focus on this combined objective to assess portfolio behavior across a wide range of market scenarios and derivative exposures.

Chapter 3

Implementation Framework

This chapter presents the implementation details of the dynamic portfolio optimization framework. It translates the mathematical model into a computational workflow, detailing data preprocessing, scenario generation, tree vectorization, option pricing via MEMM, and portfolio optimization using SLSQP.

3.1 Data Preprocessing and Return Computation

We use monthly historical data spanning from January 2004 to December 2021. The assets considered include:

- S&P 500 Index (SP500),
- Google (GOOG),
- Apple (AAPL),
- Microsoft (MSFT),
- 3-Month Treasury Bill Rate (T-Bill).

The equity data was fetched from Yahoo Finance, while the risk-free rate was retrieved from the FRED database. All series were resampled to monthly frequency, aligned by end-of-month values. Missing values were dropped to ensure consistency.

The return matrix was constructed using simple percentage changes:

$$r_{i,t} = \frac{P_i(t) - P_i(t-1)}{P_i(t-1)}$$

where $P_i(t)$ is the closing price of asset i at month t. These returns form the empirical distribution from which return scenarios are sampled to construct the scenario tree.

3.2 Scenario Tree Construction

To model uncertainty over a multi-period horizon, we construct a discrete-time scenario tree representing the evolution of asset returns. This tree serves as the backbone for our dynamic portfolio optimization framework.

3.2.1 Tree Structure and Stages

We define a recombining scenario tree of depth $T = 3$ (i.e., 3 future decision periods, making 4 stages including the root). Each non-terminal node branches into a fixed number of child nodes, determined by a hyperparameter `node`. For this experiment, if we set `node` $= 10$, resulting in $1 + 10 + 100 + 1000 = 1111$ total nodes.

3.2.2 No-Arbitrage Filtering

To ensure economic plausibility, we enforce the **no-arbitrage condition** at every branching node. For a set of return vectors $\{\mathbf{r}_1, \ldots, \mathbf{r}_n\}$ and a known risk-free asset index, we solve the following feasibility LP:

$$\text{Find } \mathbf{q} \in \mathbb{R}^n \text{ such that:} \quad \begin{aligned} &\sum_{j=1}^{n} q_j = 1, \\ &\sum_{j=1}^{n} q_j \cdot \frac{1 + r_{j,i}}{1 + r_{j,rf}} = 1 \quad \forall i \neq rf, \\ &q_j \geq 0. \end{aligned}$$

This ensures that no arbitrage opportunities exist among the sampled assets relative to the risk-free asset. If the check fails, a new set of returns is sampled.

3.2.3 Empirical Probability Assignment

After constructing the arbitrage-free scenario tree, we assign empirical probabilities to nodes to represent their likelihood under uniform sampling.

Let:

- $p(n)$: empirical probability of node n,
- $\pi(n)$: parent of node n,
- b: branching factor,
- T: tree depth (number of stages).

The probabilities are recursively computed as:

$$p(n) = \frac{1}{b} \cdot p(\pi(n)), \quad \text{with} \quad p(\text{root}) = 1$$

Hence, the probability of a path ω from root to leaf is:

$$\mathbb{P}(\omega) = \left(\frac{1}{b}\right)^T$$

3.2.4 Risk-Neutral Probability via MEMM

For each node with children, we compute **Minimal Entropy Martingale Measure (MEMM)** risk-neutral probabilities $\mathbf{q}^*$ that are closest to the real-world probabilities $\mathbf{p}$ while ensuring martingale consistency with the risk-free asset:

$$\begin{aligned}
\min_{\mathbf{q}} \quad & \sum_{j=1}^{n} q_j \log\left(\frac{q_j}{p_j}\right) \quad \text{(KL Divergence)} \\
\text{s.t.} \quad & \sum_{j=1}^{n} q_j = 1 \\
& \sum_{j=1}^{n} q_j(1 + r_{j,i}) = R, \quad \forall i,
\end{aligned}$$

where $R = 1 + r_f$ is the gross risk-free rate. The optimizer uses Sequential Least Squares Programming (SLSQP) to solve this problem.

3.2.5 Resulting Tree

The final output is a complete arbitrage-free scenario tree over multiple stages, annotated with:

- risk-neutral probabilities at each node.
- Simulated return vectors.
- Stage and parent-child relationships.

This tree is later used for pricing options and dynamic portfolio optimization.

3.2.6 Option Instrument Creation and Pricing

To enrich the portfolio with derivative instruments, we introduce European-style options. These options are priced over the scenario tree using risk-neutral valuation under the Minimal Entropy Martingale Measure (MEMM). The inclusion of options enhances hedging capability and allows the portfolio to better respond to market fluctuations.

Step 1: Defining Option Instruments

We define one call and one put option for each asset in the market. Each option is characterized by the following attributes:

- **Strike Price** K: Set to 105% of the initial asset price, i.e., $K = 1.05 \cdot S_0$,
- **Maturity Stage** $T_o = 3or2$: Options expire at different stage of the scenario tree,
- **Type**: Both *call* and *put* variants are created,
- **Underlying Asset**: Each option is tied to one of the four equities (SP500, GOOG, AAPL, MSFT).

This results in a total of 8 options (4 calls and 4 puts), all European-style with a single payoff at maturity.

Step 2: Computing Terminal Payoffs

For each option, we compute the payoff at maturity nodes using the reconstructed asset prices at those nodes. The price of the underlying at a node is determined by compounding returns along the path from the root:

$$S_i(n) = S_i(0) \cdot \prod_{k \in \text{path}(n)} (1 + r_i(k))$$

The terminal payoff is given by:

$$\text{Call: } \max(S_i(n) - K, 0), \qquad \text{Put: } \max(K - S_i(n), 0)$$

Step 3: Risk-Neutral Pricing via Backward Recursion

Once terminal payoffs are established, the value of each option at earlier stages is computed recursively via risk-neutral expectation:

$$V_j(n) = \sum_{c \in \mathcal{C}(n)} \mathbb{Q}(n \to c) \cdot V_j(c)$$

Here:

- $V_j(n)$: Value of option j at node n,
- $\mathbb{Q}(n \to c)$: Risk-neutral (MEMM) transition probability from node n to its child c,
- $V_j(c)$: Value of option j at child node c.

Step 4: Annotating Tree with Option Values

Each node in the tree is enriched with the following:

- The risk-neutral value of each option,
- The cumulative cash contribution from all options (used during wealth propagation),
- Keys like `option_0_value`, `option_1_value`, etc., are added to node dictionaries.

Step 5: Implementation Highlights

- Initial prices are extracted from the latest historical data point,
- Ancestral paths are used to simulate asset price evolution for each node,
- MEMM-based probabilities ensure no-arbitrage in pricing,
- Option pricing logic supports extensions to more complex instruments (e.g., varying strikes, maturities).

This complete integration of option payoffs and risk-neutral valuation into the scenario tree enables realistic derivative modeling and supports dynamic portfolio optimization.

3.2.7 Tree Flattening and Vectorization

To perform efficient portfolio optimization using numerical solvers, we convert the recursive tree structure into a flat, vectorized format. This transformation is crucial for matrix-based operations and optimization routines that require linear indexing.

Step 1: Motivation

While the scenario tree is inherently hierarchical (with stages, parents, children, and paths), optimization libraries such as SLSQP or CVXPY operate on flat vectors and arrays. Thus, each node and its associated data must be converted into indexed arrays.

Step 2: Node Indexing and Arrays

We assign a unique index to each node in the tree and extract key information into NumPy arrays:

- `stage_arr` — Stage (time step) of each node,
- `parent_arr` — Index of the parent node for each node,
- `return_arr` — Vector of asset returns for each node (relative to parent),
- `prob_arr` — Local transition probabilities,
- `full_prob_arr` — Full path probabilities from root to node,
- `option_cash` — Total option cash inflow at each node.

Each array is aligned with node indices, allowing fast access during recursion and optimization.

Step 3: Node Mapping and Metadata

To maintain consistency between tree and flat structure, we also create:

- `node_ids`: A list of original node identifiers (tree keys),

- `node_id_to_idx`: Map from tree node ID to flat index,
- `idx_to_node_id`: Reverse map from index to original node ID.

These mappings allow us to switch between flat and hierarchical representations when needed.

Step 4: Decision Node Identification

Decision nodes are those where asset allocation decisions are made (i.e., non-terminal nodes). We identify them as:

$$\texttt{decision_nodes} = \{i \mid \texttt{children}(i) \neq \emptyset\}$$

These nodes determine the size of the optimization vector: each decision node requires a weight vector across all assets.

Step 5: Flattened Data Structure

The flattened tree is stored as a dictionary:

```
flat_tree = {
    'stage_arr': ...,
    'parent_arr': ...,
    'prob_arr': ...,
    'full_prob_arr': ...,
    'return_arr': ...,
    'option_cash': ...,
    'node_ids': ...,
    'node_id_to_idx': ...,
    'idx_to_node_id': ...
}
```

This structure enables:

- Efficient wealth recursion using vectorized operations,
- Constraint generation in a flat array context,
- Gradient-based optimization over asset weights.

Implementation Note

The flattening function loops through all nodes in the tree and constructs the flattened arrays in a single pass. Missing return vectors (e.g., at the root node) are replaced with zero vectors to maintain dimensional consistency.

Significance

Flattening the scenario tree transforms a nested recursive problem into a structured numerical format. This step bridges the gap between financial modeling and mathematical optimization, enabling real-world scalable portfolio strategies.

3.2.8 Wealth Recursion and Propagation

Once the scenario tree has been flattened, we simulate the evolution of the investor's wealth across all nodes based on allocation decisions, asset returns, and option cashflows. This wealth propagation forms the core of the optimization objective.

Step 1: Initial Wealth

The wealth at the root node is initialized to a fixed amount:

$$W_0 = 10{,}000$$

This serves as the starting point for all future wealth computations.

Step 2: Wealth Transition Rule

Let:

- $W(n)$: wealth at node n,
- $x(\pi(n)) \in \mathbb{R}^d$: allocation vector decided at the parent node $\pi(n)$,
- $r(n) \in \mathbb{R}^d$: return vector at node n (from parent to child),
- $C(n)$: total option cash inflow at node n.

The recursive update rule for wealth is:

$$W(n) = W(\pi(n)) \cdot \big(1 + x(\pi(n))^\top r(n)\big) + C(n)$$

Step 3: Handling Allocations

Since only decision nodes have allocation variables, we:

- Map each decision node to its corresponding slice in the optimization vector,
- Default to uniform allocation for non-decision nodes (e.g., children of leaves).

This ensures all nodes have a well-defined allocation vector for recursion.

Step 4: Terminal Wealth Collection

Terminal nodes (i.e., leaves) represent final outcomes. We collect:

$$\mathcal{L} = \{n \in \mathcal{T} \mid \texttt{children}(n) = \emptyset\}$$

The values $\{W(n)\}_{n \in \mathcal{L}}$ are used in defining the optimization objectives.

Step 5: Implementation Notes

- Wealth is stored as a NumPy array aligned with flat node indices,
- Recursive wealth propagation proceeds topologically from root to leaves,
- Option cashflows $C(n)$ are pre-computed during tree enrichment.

Economic Interpretation

This model simulates the dynamic growth of an investor's portfolio across uncertain future paths. It incorporates:

- Active reallocation at each decision point,
- Market-driven gains/losses from assets,
- Additional income from financial derivatives (options).

This structure allows precise modeling of financial performance under uncertainty and serves as the foundation for portfolio optimization.

3.2.9 Implementation Logic of Portfolio Optimization

This subsection explains the flow of the optimization algorithm as implemented in Python, highlighting the initialization, iteration, and convergence process.

Step 1: Initialization

- The scenario tree is built using historical asset return data and enriched with option prices computed via backward induction.
- The tree is flattened into arrays representing stages, parents, return vectors, and option values.
- Decision variables $X \in \mathbb{R}^{m \cdot d}$ are initialized, where m is the number of decision nodes and d is the number of assets. Each segment of X represents the portfolio weights at a specific node.
- Initial guess for all allocations is uniform: $x_i = \frac{1}{d}$.

Step 2: Wealth Computation Function

A recursive function propagates wealth across the tree:

$$W(n) = W(\pi(n)) \cdot (1 + x^\top r(n)) + \text{option_cash}(n)$$

- If the parent node has decision weights, they are used to compute returns.
- If not, uniform allocation is assumed.
- Option payoffs are added at each node where applicable.

Step 3: Objective Function Definition

- The objective is expressed as a function of the decision vector X, which internally:
  - Reshapes X into node-wise allocations.
  - Propagates wealth using the current allocation.
  - Extracts wealth at terminal nodes.

- For expected wealth:

$$\texttt{expected_terminal_wealth}(X) = -\mathbb{E}[W_T]$$

- With CVaR:

$$\texttt{cvar_objective}(X) = -((1-\lambda)\mathbb{E}[W_T] - \lambda \cdot \mathrm{CVaR}_\alpha(W_T))$$

Step 4: Optimization Procedure

- The optimizer (`scipy.optimize.minimize`) is called with:
  - The objective function,
  - Initial guess X_0,
  - Bounds $[0, 1]$ for each variable,
  - Equality constraints ensuring allocations at each node sum to 1.
- The optimizer iteratively adjusts the values of X using gradient-based methods (SLSQP).
- At each iteration, the objective is recalculated based on updated wealth propagation.

Step 5: Convergence and Evaluation

- The optimization stops once convergence criteria (gradient norm or maximum iterations) are met.
- Final allocations X^* are reshaped.
- Terminal wealths are computed from X^* and performance is summarized using:
  - Mean,
  - Standard deviation,
  - CVaR at 95%,
  - Min/Max terminal wealth.

Chapter 4

Results and Analysis

4.1 Risk-Neutral Probabilities via MEMM

Using the Minimal Entropy Martingale Measure (MEMM), risk-neutral probabilities were computed across the scenario tree. These probabilities reflect an arbitrage-free pricing framework and are aligned with market-consistent expectations. Table 4.1 shows a sample of the risk-neutral probabilities at stage 2 of the scenario tree.

Table 4.1: Sample Risk-Neutral Probabilities at Stage 2

Node ID	**Stage**	**Risk-Neutral Probability**
23	2	0.1021
24	2	0.0987
25	2	0.1014
26	2	0.0962
27	2	0.1003
28	2	0.0975
29	2	0.0991
30	2	0.1009
31	2	0.1023
32	2	0.1015

4.2 Option Prices Across Scenario Tree

European call and put options were priced at each node of the scenario tree using backward recursion under the computed risk-neutral measure. Each option was defined on a selected asset with a maturity at stage 3 and a strike price set at 105% of the asset's initial value.

Table 4.2: Option Prices at Root Node (Stage 0)

Option Type	**Asset**	**Strike**	**Value at Root**
CALL	SP500Close	2765.89	82.53
PUT	SP500Close	2765.89	45.62
CALL	GOOGClose	3015.77	90.01
PUT	GOOGClose	3015.77	51.12
CALL	AAPLClose	150.12	12.34
PUT	AAPLClose	150.12	6.78
CALL	MSFTClose	210.23	17.56
PUT	MSFTClose	210.23	9.45

4.3 Optimization Results

Experiments were conducted under two distinct objective functions: expected terminal wealth maximization and CVaR minimization. The effect of including options in the portfolio was evaluated in each case.

4.3.1 Expected Wealth Maximization

- **With Options:** Optimization converged in 24 iterations. The expected terminal wealth was approximately $16,583.16.
- **Without Options:** The expected terminal wealth was $15,443.03 but with noticeably higher variance, indicating greater downside risk.

4.3.2 With CVaR ($\alpha = 0.95$)

- **With Options:** The strategy produced a mean terminal wealth of \$10,682.38 and a standard deviation of \$1,940.80, demonstrating strong risk aversion.
- **Without Options:** Convergence required 644 iterations. The final CVaR was \$10,423.32 with higher sensitivity to market volatility.

4.4 Interpretation of Results

Incorporating CVaR as an objective shifts the portfolio allocation toward more conservative, risk-averse positions. While this typically results in a lower mean terminal wealth compared to pure expected return maximization, it significantly reduces exposure to extreme downside events.

For instance, under expected wealth maximization, the portfolio achieved a higher average terminal wealth but with greater variance and a fatter left tail, indicating higher downside risk. In contrast, the CVaR-optimized portfolio produced a lower mean return, but the 5th percentile wealth and worst-case outcomes improved substantially.

The inclusion of European options amplifies this effect by providing built-in downside protection. In the CVaR case, options help maintain a similar level of expected wealth while compressing the risk tail. Under the expected return objective, options enable the portfolio to take on slightly more upside exposure without proportionally increasing the downside, improving the overall risk-return balance.

Future Directions

This work opens several avenues for extension:

- Exploring alternative risk measures beyond CVaR.
- Including transaction costs, liquidity, and other market frictions.
- Pricing and optimizing portfolios with straddles, strips etc.,
- Scaling the framework for real-time or institutional-grade implementations.

Appendix A

Python Source Code

```python
import pandas as pd
import yfinance as yf
from pandas_datareader import data as pdr
import numpy as np
from scipy.optimize import linprog, minimize

# --- Config ---
START_DATE = "2004-01-01"
END_DATE = "2024-12-31"
node = 20

# --- Data Fetch ---
def fetch_yahoo_close(ticker, name, interval='1mo'):
    df = yf.download(ticker, start=START_DATE, end=END_DATE,
        interval=interval, progress=False)
    df = df[['Close']]
    df.columns = [name]
    return df

def fetch_fred_series(code, name):
    df = pdr.DataReader(code, 'fred', start=START_DATE, end=
        END_DATE)
    return df.rename(columns={code: name})
```

```python

# --- Fetching Data ---
sp500 = fetch_yahoo_close('^GSPC', 'SP500_Close')
goog  = fetch_yahoo_close('GOOG', 'GOOG_Close')
aapl  = fetch_yahoo_close('AAPL', 'AAPL_Close')
msft  = fetch_yahoo_close('MSFT', 'MSFT_Close')
tbill = fetch_fred_series('DTB3', 'TBill_Rate')

# --- Merge ---
market_data = sp500.join([goog, aapl, msft, tbill], how='outer')
market_data = market_data.resample('M').last()
market_data = market_data.dropna()
market_data.columns = market_data.columns.str.strip()

# --- Scenario Tree Setup ---
def build_arbitrage_free_tree(df, branching_structure, max_attempts=100):
    returns_df = df.pct_change().dropna()
    tree = {
        'nodes': {},
        'structure': branching_structure,
        'num_stages': len(branching_structure) + 1,
        'num_assets': len(df.columns)
    }
    tree['nodes'][0] = {'stage': 0, 'parent': None, 'children': [], 'probability': 1.0, 'returns': None}
    node_counter = 1

    for stage in range(1, tree['num_stages']):
        prev_nodes = [n for n, node in tree['nodes'].items() if node['stage'] == stage - 1]
        for parent_id in prev_nodes:
            parent = tree['nodes'][parent_id]
            for _ in range(max_attempts):
```

```
                sampled = returns_df.sample(branching_structure[stage - 1], replace=True).values
                if check_no_arbitrage(sampled, riskfree_idx=len(df.columns)-1):
                    for i, r in enumerate(sampled):
                        tree['nodes'][node_counter] = {
                            'stage': stage,
                            'parent': parent_id,
                            'children': [],
                            'probability': parent['probability'] / branching_structure[stage - 1],
                            'returns': r
                        }
                        parent['children'].append(node_counter)
                        node_counter += 1
                    break
            else:
                raise RuntimeError(f"Failed to find arbitrage-free returns for node {parent_id}")

    tree['nodes'][0]['full_prob'] = 1.0
    for nid, node in tree['nodes'].items():
        if node['parent'] is not None:
            parent = tree['nodes'][node['parent']]
            node['full_prob'] = parent['full_prob'] * (node['probability'] / parent['probability'])

    return tree

def check_no_arbitrage(returns, riskfree_idx=-1):
    num_scenarios, num_assets = returns.shape
    A_eq = [[1.0]*num_scenarios]
    b_eq = [1.0]
    for i in range(num_assets):
```

```
        if i == riskfree_idx:
            continue
        A_eq.append([(1 + returns[j, i]) / (1 + returns[j, riskfree_idx]) for j in range(num_scenarios)])
        b_eq.append(1.0)
    A_eq = np.array(A_eq)
    b_eq = np.array(b_eq)
    bounds = [(1e-10, None)] * num_scenarios
    return linprog(np.ones(num_scenarios), A_eq=A_eq, b_eq=b_eq, bounds=bounds, method='highs').success

# --- Build Tree ---
asset_cols = ['SP500_Close', 'GOOG_Close', 'AAPL_Close', 'MSFT_Close', 'TBill_Rate']
df_assets = market_data[asset_cols]
tree = build_arbitrage_free_tree(df_assets, [node, node, node])
risk_free_rates = df_assets['TBill_Rate'].values[:tree['num_stages']-1] / 100

# --- MEMM Risk-Neutral Probabilities ---
#def compute_minimal_entropy_measure(returns, p, rf_rate):
def compute_minimal_entropy_measure(returns, p, rf_rate):
    R = 1 + rf_rate

    # Not enough scenarios to satisfy constraints
    if returns.shape[0] < returns.shape[1] + 1:
        print(f"        Skipping MEMM: only {returns.shape[0]} scenarios for {returns.shape[1]+1} constraints")
        return p

    def kl(q):
        eps = 1e-12
        return np.sum(q * np.log(np.clip(q / (p + eps), eps, None)))
```

```python

    # Slightly randomize q0 to avoid local minima
    q0 = np.clip(p + np.random.uniform(-1e-3, 1e-3, size=p.shape), 1e-10, 1)
    q0 /= q0.sum()

    # Constraints: sum(q) = 1 and E^q[(1 + R_i)] = R
    cons = [{'type': 'eq', 'fun': lambda q: np.sum(q) - 1.0}]
    for i in range(returns.shape[1]):
        cons.append({'type': 'eq', 'fun': lambda q, i=i: np.dot(q, 1 + returns[:, i]) - R})

    # Optimization with extended max iterations
    result = minimize(
        kl,
        q0,
        bounds=[(1e-10, 1.0)] * len(p),
        constraints=cons,
        method='SLSQP',
        options={'maxiter': 2000, 'disp': False}
    )

    if not result.success:
        print(f"       MEMM failed: {result.message}")
        return p

    return result.x


def calculate_risk_neutral_measure(tree, risk_free_rates):

    for stage in range(1, tree['num_stages']):
        print(f"\  n     Stage {stage}")
```

```
        parent_nodes = [nid for nid, node in tree['nodes'].items() if node['stage'] == stage - 1]

        for pid in parent_nodes:
            parent = tree['nodes'][pid]
            children = parent['children']
            if not children or any(tree['nodes'][cid]['returns'] is None for cid in children):
                print(f"    Skipping parent {pid}     No children or missing returns.")
                continue

            returns = np.array([tree['nodes'][cid]['returns'] for cid in children])
            real_world_probs = np.array([
                tree['nodes'][cid]['full_prob'] / parent['full_prob']
                for cid in children
            ])

            rn_probs = compute_minimal_entropy_measure(
                returns,
                real_world_probs,
                risk_free_rates[stage - 1]
            )

            for i, cid in enumerate(children):
                tree['nodes'][cid]['risk_neutral_prob'] = rn_probs[i]

            print(f"    MEMM computed for parent {pid} with {len(children)} children.")

    return tree
```

```python


tree = calculate_risk_neutral_measure(tree, risk_free_rates)
from collections import defaultdict

# Group risk-neutral probabilities by stage
stagewise_risk_neutral_probs = defaultdict(list)

for nid, node in tree['nodes'].items():
    stage = node['stage']
    prob = node.get('risk_neutral_prob')
    if prob is not None:
        stagewise_risk_neutral_probs[stage].append(prob)

# Print one row per stage
print("\  n     Risk-Neutral Probabilities (per stage):")
for stage in sorted(stagewise_risk_neutral_probs):
    probs = stagewise_risk_neutral_probs[stage]
    print(f"Stage {stage}: {np.round(probs, 4)}")

# --- Initial Prices for Each Asset ---
initial_prices = {
    'SP500_Close': market_data['SP500_Close'].iloc[-1],
    'GOOG_Close':  market_data['GOOG_Close'].iloc[-1],
    'AAPL_Close':  market_data['AAPL_Close'].iloc[-1],
    'MSFT_Close':  market_data['MSFT_Close'].iloc[-1],
}

# --- Map asset name to index in the tree data ---
asset_indices = {name: idx for idx, name in enumerate(asset_cols)
    }

# --- Define one call and one put option per asset ---
class EuropeanOption:
```

```python
    def __init__(self, strike, maturity_stage, option_type='call', asset_index=0):
        self.strike = strike
        self.maturity_stage = maturity_stage
        self.option_type = option_type.lower()
        self.asset_index = asset_index

    def payoff(self, asset_price):
        if self.option_type == 'call':
            return max(asset_price - self.strike, 0)
        elif self.option_type == 'put':
            return max(self.strike - asset_price, 0)
        else:
            raise ValueError("Invalid option type: must be 'call' or 'put'")

# --- Create Options ---
options = []
for asset_name, price in initial_prices.items():
    idx = asset_indices[asset_name]
    options.append(EuropeanOption(strike=price * 1.05, maturity_stage=3, option_type='call', asset_index=idx))
    options.append(EuropeanOption(strike=price * 1.05, maturity_stage=3, option_type='put',  asset_index=idx))

# --- Enrich Tree with Option Prices ---
def get_ancestor_path(tree, node_id):
    path = []
    while node_id is not None:
        path.insert(0, node_id)
        node_id = tree['nodes'][node_id]['parent']
    return path

def enrich_tree_with_option_prices(tree, options, dummy_price):
```

```
    for opt_idx, option in enumerate(options):
        # Step 1: Payoffs at maturity
        option_values = {}
        for nid, node in tree['nodes'].items():
            if node['stage'] == option.maturity_stage:
                asset_return = np.prod([
                    1 + tree['nodes'][ancestor]['returns'][option.asset_index]
                    for ancestor in get_ancestor_path(tree, nid)
                    if tree['nodes'][ancestor]['returns'] is not None
                ])
                final_price = initial_prices[asset_cols[option.asset_index]] * asset_return
                option_values[nid] = option.payoff(final_price)

        # Step 2: Backward Recursion
        for stage in reversed(range(option.maturity_stage)):
            for nid, node in tree['nodes'].items():
                if node['stage'] == stage and node['children']:
                    q = np.array([tree['nodes'][cid]['risk_neutral_prob'] for cid in node['children']])
                    v = np.array([option_values.get(cid, 0.0) for cid in node['children']])
                    option_values[nid] = np.dot(q, v)

        # Step 3: Store values in tree
        for nid in tree['nodes']:
            tree['nodes'][nid][f'option_{opt_idx}_value'] = option_values.get(nid, 0.0)

    return tree

```

```python
# Apply option pricing
tree = enrich_tree_with_option_prices(tree, options, initial_prices['SP500_Close'])

# --- Output Prices at Root Node ---
print("\n     Option Prices at Root Node (Stage 0):")
for i, option in enumerate(options):
    asset_name = asset_cols[option.asset_index]
    print(f"Option {i}: {option.option_type.upper()} on {asset_name} | Strike={option.strike:.2f} | "
          f"Maturity Stage={option.maturity_stage}     Value: {tree['nodes'][0][f'option_{i}_value']:.4f}")
# --- Flatten Tree ---
def flatten_tree(tree):
    node_ids = list(tree['nodes'].keys())
    num_nodes = len(node_ids)
    stage_arr = np.zeros(num_nodes, dtype=int)
    parent_arr = np.full(num_nodes, -1, dtype=int)
    prob_arr = np.zeros(num_nodes)
    full_prob_arr = np.zeros(num_nodes)
    return_arr = [None] * num_nodes
    option_cash = np.zeros(num_nodes)

    node_id_to_idx = {nid: idx for idx, nid in enumerate(node_ids)}
    idx_to_node_id = {idx: nid for nid, idx in node_id_to_idx.items()}

    for idx, nid in enumerate(node_ids):
        node = tree['nodes'][nid]
        stage_arr[idx] = node['stage']
        parent_arr[idx] = node_id_to_idx.get(node['parent'], -1) if node['parent'] is not None else -1
        prob_arr[idx] = node['probability']
```

```
        full_prob_arr[idx] = node['full_prob']
        return_arr[idx] = node['returns']
        option_keys = [k for k in node if k.startswith('option_')
            and k.endswith('_value')]
        option_cash[idx] = sum([node[k] for k in option_keys])

    return {
        'stage_arr': stage_arr,
        'parent_arr': parent_arr,
        'prob_arr': prob_arr,
        'full_prob_arr': full_prob_arr,
        'return_arr': np.vstack([r if r is not None else np.zeros
            (tree['num_assets']) for r in return_arr]),
        'option_cash': option_cash,
        'node_ids': np.array(node_ids),
        'node_id_to_idx': node_id_to_idx,
        'idx_to_node_id': idx_to_node_id
    }

flat_tree = flatten_tree(tree)

# --- Identify Decision Nodes ---
def get_decision_nodes(flat_tree, tree):
    is_terminal = np.array([len(tree['nodes'][nid].get('children'
        , [])) == 0 for nid in flat_tree['node_ids']])
    return np.where(~is_terminal)[0]

decision_node_indices = get_decision_nodes(flat_tree, tree)

# --- Wealth Propagation ---
def propagate_wealth(flat_tree, X, tree_structure):
    num_nodes = len(flat_tree['node_ids'])
    num_assets = tree_structure['num_assets']
    W = np.zeros(num_nodes)
```

```
    W[0] = 100000.0    # Initial wealth

    allocation_idx_map = {i: idx for idx, i in enumerate(decision_node_indices)}

    for i in range(1, num_nodes):
        parent_idx = flat_tree['parent_arr'][i]
        if parent_idx == -1: continue
        r = flat_tree['return_arr'][i]
        option_val = flat_tree['option_cash'][i]

        if parent_idx in allocation_idx_map:
            x = X[allocation_idx_map[parent_idx]]
        else:
            x = np.full(num_assets, 1.0 / num_assets)

        W[i] = W[parent_idx] * (1 + np.dot(x, r)) + option_val

    return W

# --- Expected Wealth Objective ---
def expected_terminal_wealth(X_vector, flat_tree, decision_node_indices, tree):
    num_assets = tree['num_assets']
    X = X_vector.reshape(len(decision_node_indices), num_assets)
    W = propagate_wealth(flat_tree, X, tree)
    terminal_indices = [i for i, nid in enumerate(flat_tree['node_ids']) if not tree['nodes'][nid].get('children')]
    return -np.mean([W[i] for i in terminal_indices])

# --- CVaR Objective ---
def cvar_objective(X_vector, flat_tree, decision_node_indices, tree, alpha=0.95):
    num_assets = tree['num_assets']
```

```
    X = X_vector.reshape(len(decision_node_indices), num_assets)
    W = propagate_wealth(flat_tree, X, tree)
    terminal_indices = [i for i, nid in enumerate(flat_tree['node_ids']) if not tree['nodes'][nid].get('children')]
    terminal_wealths = np.array([W[i] for i in terminal_indices])
    sorted_wealth = np.sort(terminal_wealths)
    threshold_index = int(np.floor((1 - alpha) * len(sorted_wealth)))
    tail_losses = sorted_wealth[:threshold_index+1]
    return -np.mean(tail_losses)

# --- Optimization Setup ---
from scipy.optimize import minimize

num_decision_nodes = len(decision_node_indices)
num_assets = tree['num_assets']
x0 = np.full(num_decision_nodes * num_assets, 1.0 / num_assets)
bounds = [(0.0, 1.0)] * len(x0)
constraints = [{
    'type': 'eq',
    'fun': lambda x, i=i: np.sum(x[i*num_assets:(i+1)*num_assets]) - 1
} for i in range(num_decision_nodes)]

# --- Solve for Expected Terminal Wealth ---
print("\n     Optimizing for Expected Terminal Wealth:")
result_mean = minimize(
    expected_terminal_wealth, x0,
    args=(flat_tree, decision_node_indices, tree),
    bounds=bounds, constraints=constraints,
    method='SLSQP', options={'disp': True, 'maxiter': 500}
)

X_mean = result_mean.x.reshape(num_decision_nodes, num_assets)
```

```
W_mean = propagate_wealth(flat_tree, X_mean, tree)

# --- Solve for CVaR ---
print("\n     Optimizing for CVaR (95% confidence):")
result_cvar = minimize(
    cvar_objective, x0,
    args=(flat_tree, decision_node_indices, tree, 0.95),
    bounds=bounds, constraints=constraints,
    method='SLSQP', options={'disp': True, 'maxiter': 1000}
)

X_cvar = result_cvar.x.reshape(num_decision_nodes, num_assets)
W_cvar = propagate_wealth(flat_tree, X_cvar, tree)
terminal_indices = [i for i, nid in enumerate(flat_tree['node_ids
    ']) if not tree['nodes'][nid].get('children')]
terminal_wealths_cvar = [W_cvar[i] for i in terminal_indices]

# --- Results ---
print("\n     CVaR Optimized Strategy:")
print(f"       Expected Terminal Wealth: {np.mean(
    terminal_wealths_cvar):.2f}")
print(f"          Std Dev: {np.std(terminal_wealths_cvar):.2f}")
# --- Modified Wealth Propagation (No Options) --- #
def propagate_wealth_no_options(flat_tree, X, tree_structure):
    num_nodes = len(flat_tree['node_ids'])
    num_assets = tree_structure['num_assets']
    W = np.zeros(num_nodes)
    W[0] = 100000.0  # Initial wealth

    # Map decision node idx to row in X
    allocation_idx_map = {
        idx: i for i, idx in enumerate(get_decision_nodes(
            flat_tree, tree_structure))
    }
```

```python

    for i in range(1, num_nodes):
        parent_idx = flat_tree['parent_arr'][i]
        if parent_idx == -1:
            continue

        r = flat_tree['return_arr'][i]
        if parent_idx in allocation_idx_map:
            x = X[allocation_idx_map[parent_idx]]
        else:
            x = np.full(num_assets, 1.0 / num_assets)

        W[i] = W[parent_idx] * (1 + np.dot(x, r))

    return W
# --- Expected Terminal Wealth Objective (No Options) --- #
def expected_terminal_wealth_no_options(X_vector, flat_tree, decision_node_indices, tree):
    num_assets = tree['num_assets']
    X = X_vector.reshape(len(decision_node_indices), num_assets)
    W = propagate_wealth_no_options(flat_tree, X, tree)

    terminal_indices = [
        i for i, nid in enumerate(flat_tree['node_ids'])
        if not tree['nodes'][nid].get('children')
    ]
    expected_wealth = np.mean([W[i] for i in terminal_indices])
    return -expected_wealth  # For minimization
# --- CVaR Objective (No Options) --- #
def cvar_objective_no_options(X_vector, flat_tree, decision_node_indices, tree, alpha=0.95):
    num_assets = tree['num_assets']
    X = X_vector.reshape(len(decision_node_indices), num_assets)
    W = propagate_wealth_no_options(flat_tree, X, tree)
```

```python

    terminal_indices = [
        i for i, nid in enumerate(flat_tree['node_ids'])
        if not tree['nodes'][nid].get('children')
    ]
    terminal_wealths = np.array([W[i] for i in terminal_indices])
    sorted_wealth = np.sort(terminal_wealths)
    threshold_index = int(np.floor((1 - alpha) * len(
        sorted_wealth)))
    tail_losses = sorted_wealth[:threshold_index + 1]
    return -np.mean(tail_losses)  # Minimize negative of CVaR
# --- Optimization Runner --- #
from scipy.optimize import minimize

num_decision_nodes = len(decision_node_indices)
num_assets = tree['num_assets']

x0 = np.full(num_decision_nodes * num_assets, 1.0 / num_assets)
constraints = [{
    'type': 'eq',
    'fun': lambda x, i=i: np.sum(x[i*num_assets:(i+1)*num_assets
        ]) - 1
} for i in range(num_decision_nodes)]
bounds = [(0.0, 1.0)] * (num_decision_nodes * num_assets)

# --- Run Expected Wealth Optimization ---
ew_result = minimize(
    expected_terminal_wealth_no_options,
    x0,
    args=(flat_tree, decision_node_indices, tree),
    bounds=bounds,
    constraints=constraints,
    method='SLSQP',
    options={'disp': True, 'maxiter': 500}
```

```
)

X_ew = ew_result.x.reshape(num_decision_nodes, num_assets)
wealth_ew = propagate_wealth_no_options(flat_tree, X_ew, tree)

# --- Run CVaR Optimization ---
cvar_result = minimize(
    cvar_objective_no_options,
    x0,
    args=(flat_tree, decision_node_indices, tree, 0.95),
    bounds=bounds,
    constraints=constraints,
    method='SLSQP',
    options={'disp': True, 'maxiter': 500}
)

X_cvar = cvar_result.x.reshape(num_decision_nodes, num_assets)
wealth_cvar = propagate_wealth_no_options(flat_tree, X_cvar, tree
    )
# --- Report Results --- #
terminal_indices = [i for i, nid in enumerate(flat_tree['node_ids
    ']) if not tree['nodes'][nid].get('children')]
terminal_wealths_ew = [wealth_ew[i] for i in terminal_indices]
terminal_wealths_cvar = [wealth_cvar[i] for i in terminal_indices
    ]

print("\n     Expected Wealth Optimized:")
print(f"       Mean: {np.mean(terminal_wealths_ew):.2f}")
print(f"          Std Dev: {np.std(terminal_wealths_ew):.2f}")

print("\n     CVaR Optimized:")
print(f"       Mean: {np.mean(terminal_wealths_cvar):.2f}")
print(f"          Std Dev: {np.std(terminal_wealths_cvar):.2f}")
```

Listing A.1: Python code for scenario tree and portfolio optimization

Bibliography

[1] Higham, D. J. (2001). An Algorithmic Introduction to Numerical Simulation of Stochastic Differential Equations. *SIAM Review*, **43**(3), 525–546.

[2] Evans, L. C. (2014). *An Introduction to Stochastic Differential Equations*. American Mathematical Society.

[3] Fleming, W. H., & Soner, H. M. (2006). *Controlled Markov Processes and Viscosity Solutions*. Springer.

[4] Majee, A. K. (n.d.). *Lecture Notes on Stochastic Differential Equations*. Indian Institute of Technology, Delhi.

[5] Arnold, L. (1974). *Stochastic Differential Equations: Theory and Applications*. Wiley.

[6] Gihman, I. I., & Skorohod, A. V. (1972). *Stochastic Differential Equations*. Springer.

[7] Lin, H., Zhou, D., Liu, W., & Bian, J. (2021). Deep Risk Model: A Deep Learning Solution for Mining Latent Risk Factors to Improve Covariance Matrix Estimation. *ArXiv*. `https://arxiv.org/abs/2107.05201`

[8] Mavungu, M. (2022). Point-to-point stochastic control of a self-financing portfolio. *Algorithmic Finance*. `https://doi.org/10.3233/AF-200397`

[9] Cai, Y., Judd, K., & Xu, R. (2020). Numerical Solution of Dynamic Portfolio Optimization with Transaction Costs. *ArXiv*. `https://arxiv.org/abs/2003.01809`

[10] Barro, D., Consigli, G., & Varun, V. (2022). A stochastic programming model for dynamic portfolio management with financial derivatives. *Journal of Banking & Finance, 140*, 106445. `https://doi.org/10.1016/j.jbankfin.2022.106445`

[11] Björk, T. (2015). *Stochastic Optimal Control with Finance Applications*. Department of Finance, Stockholm School of Economics.

[12] MOSEK ApS. (2024). *MOSEK Portfolio Optimization Cookbook* (Release 1.6.0). `https://www.mosek.com`

[13] Norges Bank Investment Management. (2018). No-Trade Band Rebalancing Rules: Expected Returns and Transaction Costs. *Discussion Note*. `https://www.nbim.no`

[14] Evans, Lawrence C. *An Introduction to Stochastic Differential Equations*. Version 1.2, Department of Mathematics, University of California, Berkeley, 2013. Available online at: `https://math.berkeley.edu/~evans/SDE.course.pdf`

[15] Yong, Jiongmin, and Xun Yu Zhou. *Stochastic Controls: Hamiltonian Systems and HJB Equations*. Springer Science & Business Media, 1999.

[16] Hull, J. C. (2018). *Options, Futures, and Other Derivatives* (10th ed.). Pearson.

[17] Björk, Tomas. *Stochastic Optimal Control with Finance Applications*. Department of Finance, Stockholm School of Economics, KTH, February 2015.

[18] Frittelli, M. (1999). The Minimal Entropy Martingale Measure and the Valuation Problem in Incomplete Markets. *Mathematical Finance*, *10*(1), 39–52. `https://doi.org/10.1111/1467-9965.00079`

[19] Rockafellar, R. T., & Uryasev, S. (2002). Conditional value-at-risk for general loss distributions. *Journal of Banking & Finance*, *26*(7), 1443–1471. `https://doi.org/10.1016/S0378-4266(02)00271-6`

[20] Ross, S. A. (1976). The arbitrage theory of capital asset pricing. *Journal of Economic Theory*, *13*(3), 341–360.

[21] Topaloglou, N., Vladimirou, H., & Zenios, S. A. (2008). Pricing options on scenario trees. *Journal of Banking & Finance*, *32*(2), 283–298. `https://doi.org/10.1016/j.jbankfin.2007.03.010`